

Fourier Coefficients Evaluation using Python

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import quad
```

```
In [2]: # function
def f(x):
    # if (0<=x<np.pi):
    #     return 1
    # else:
    #     return -1
    val = np.where(0<=x<np.pi, 1, -1)
    return val

# half period
L = np.pi
# no of fourier terms
N = 50
```

```
In [3]: # fourier coeff
def a0():
    val, _ = quad(lambda x: f(x), -L, L)
    return val/L

def an(n):
    val, _ = quad(lambda x: f(x)*np.cos(n*np.pi*x/L), -L, L)
    return val/L

def bn(n):
    val, _ = quad(lambda x: f(x)*np.sin(n*np.pi*x/L), -L, L)
    return val/L
```

```
In [4]: # value of coeff
a0_val = a0()
a_val = [an(n) for n in range(1, N + 1)]
b_val = [bn(n) for n in range(1, N + 1)]

print(a0_val, a_val, b_val)
```

```

0.0 [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
0.0, 0.0, 0.0, 0.0, 0.0, 0.0] [1.2732395447351625, 1.0899529502209053e-16,
0.42441318157838764, 1.0051089433872239e-16, 0.2546479089470324, 3.5177479
872902253e-16, 0.18189136353359414, -5.43818605928925e-16, 0.1414710605261
2907, -1.6648293491582657e-17, 0.11574904952137845, 2.7683501325461324e-1
6, 0.09794150344116648, 2.4299559926405767e-16, 0.08488263631567741, -2.00
43906416344592e-16, 0.07489644380794998, 1.2878475573665222e-15, 0.0670126
0761764039, -7.673606049974989e-16, 0.06063045451119861, 1.376517548202299
6e-15, 0.05535824107544269, -8.783295297820349e-17, 0.05092958178940592, -
3.318181101501594e-15, 0.04715702017537593, -7.595121593451756e-16, 0.0439
0481188741899, 3.500901740863332e-15, 0.04107224337855403, -2.247392144693
868e-15, 0.03858301650712719, 2.2338405180075814e-15, 0.0363782727067197,
3.650256589314776e-17, 0.03441187958743682, -1.4964606570216964e-15, 0.032
647167813722816, -3.461840819268955e-16, 0.031054623042321733, -3.06221505
86978577e-15, 0.029610221970585066, 1.8740076947629815e-15, 0.028294212105
22573, 2.2589901562935615e-15, 0.02709020307947225, -1.1975274783670268e-1
5, 0.025984480504799127, 3.7723844471606206e-15]

```

```

In [5]: # Reconstruct functions
x_vals = np.linspace(0, 2*L, 1000)
f_fourier = np.full_like(x_vals, a0_val/2)

for n in range(1, N+1):
    f_fourier += a_val[n-1]*np.cos(n*np.pi*x_vals/L) + b_val[n-1]*np.sin(

f_original = np.array([f(x) for x in x_vals])

```

```

In [6]: plt.plot(x_vals, f_original, label="Original f(x)")
plt.plot(x_vals, f_fourier, label = f"Fourier Approx (N = {N})")
plt.legend()
plt.grid(True)
plt.show()

```

